Repository File Map (`FILE_INDEX.md`)

This document provides a comprehensive structural mapping of the **Kazma Agent Framework** codebase. Kazma is a multi-platform AI agent framework featuring a LangGraph supervisor brain, swarm orchestration engine, multi-channel platform gateway (Telegram/Discord/Slack/Web/TUI), IDE integration subsystem, and an OpenAI-compatible multi-provider LLM abstraction layer.

---

1. Core Architecture & Global Configuration

`kazma.yaml`

- **Role:** Main Application Configuration File
- **Architecture Links:** Loaded by `kazma_core/config_loader.py` and consumed by `kazma_core/config_store.py` across all packages.
- **Core Responsibilities & Operations:** Defines system defaults including model profiles, provider API endpoints, active workspace paths, HITL safety approval lists, swarm worker configurations, vector store settings, and channel gateway tokens.

`pyproject.toml`

- **Role:** Monorepo Workspace Build & Package Definition
- **Architecture Links:** Governs package builds for `kazma-core`, `kazma-gateway`, `kazma-ui`, `kazma-tui`, `kazma-cli`, `kazma-memory`, and `kazma-skills`.
- **Core Responsibilities & Operations:** Specifies project dependencies, Python version requirements (>=3.11), build tools (`hatchling`), CLI entry points, and workspace package mappings.

`Dockerfile`

- **Role:** Production Container Build Descriptor
- **Architecture Links:** Packaging boundary for deployment across Docker, Docker Compose, Fly.io, and Kubernetes clusters.
- **Core Responsibilities & Operations:** Configures multi-stage build environments, installs system dependencies (`ffmpeg` , `git` , `curl`), installs Python dependencies via `uv` , exposes default ports (`8000`), and sets up execution entry points.

`docker-compose.yml`

- **Role:** Local Multi-Container Orchestration File
- **Architecture Links:** Connects the `kazma-ui` web app, `kazma-gateway` bot service, PostgreSQL instance, and vector database services.

- **Core Responsibilities & Operations:** Defines service dependencies, persistent volumes, environment variable passes, health checks, and container network bridges for local development and testing.

`serve.py`

- **Role:** Universal Production Web Server Launcher
- **Architecture Links:** Directly invokes `kazma_ui/app.py` via `uvicorn`.
- **Core Responsibilities & Operations:** Initializes environment configs, sets up host binding and port parameters, configures SSL termination options, and manages graceful startup and shutdown loops.

`setup.ps1`

- **Role:** Windows Environment Bootstrap Script
- **Architecture Links:** Sets up virtual environments (`.venv`), installs all local editable package wheels, and verifies local CLI access.
- **Core Responsibilities & Operations:** Installs `uv`, creates Python virtual environment, installs monorepo packages in editable mode (`-e`), copies template configurations, and validates dependency constraints.

`setup.sh`

- **Role:** Linux/macOS Environment Bootstrap Script
- **Architecture Links:** Shell equivalent to `setup.ps1` for POSIX development and deployment environments.
- **Core Responsibilities & Operations:** Verifies Python 3.11+, bootstraps `.venv`, installs monorepo packages editable via `uv`, and ensures runtime directory structure permissions.

`run.sh`

- **Role:** Multi-Service Startup Runner
- **Architecture Links:** Orchestrates execution of `kazma-ui`, `kazma-gateway`, and background memory/cron services.
- **Core Responsibilities & Operations:** Parses launch flags (`--web`, `--gateway`, `--tui`), manages process backgrounding, handles PID tracking, and intercepts SIGINT/SIGTERM for clean shutdown.

`swarm_templates.json`

- **Role:** Swarm Worker Blueprint Specification
- **Architecture Links:** Read by `kazma_core/swarm/autoscaler.py` during dynamic worker auto-spawning.
- **Core Responsibilities & Operations:** Stores pre-configured worker templates (Coder, Researcher, Generalist) specifying worker capabilities, expertise tags, tool sets, and prompt strategies for runtime scaling.

2. Kazma Core Subsystem (`kazma-core/kazma_core/`)

2.1 Agent Brain & LangGraph State Engine

> **`kazma-core/kazma_core/agent_runner.py`**

- **Role:** Supervisor Execution & Graph Pipeline Runner
- **Architecture Links:** Connects platform input handlers in `kazma-gateway` and `kazma-ui/sse_chat.py` to `kazma_core/graph_builder.py`.
- **Core Responsibilities & Operations:** Constructs and compiles supervisor graph instances, injects `SnapshotRecorder` for time-travel, manages graph state streaming, and handles turn initialization and response extraction.

> **`kazma-core/kazma_core/graph_builder.py`**

- **Role:** LangGraph Supervisor State Machine Builder
- **Architecture Links:** Interacts with `llm_provider.py`, `tools/registry.py`, `safety/hitl.py`, `time_travel.py`, and `skills/self_improvement.py`.
- **Core Responsibilities & Operations:** Defines graph nodes (`supervisor`, `tool_worker`, `respond_node`), builds supervisor conditional edges, handles turn retries, implements HITL interrupts for danger tools, and records state snapshots.

> **`kazma-core/kazma_core/graph.py`**

- **Role:** Chat Interface Command Resolver & Execution Router
- **Architecture Links:** Intercepts incoming messages from gateways and UI before graph execution; routes slash commands (`/ide`, `/swarm`, `/undo`, `/replay`, `/fork`).
- **Core Responsibilities & Operations:** Evaluates message prefixes, resolves slash commands, dispatches graph state rewinds or forks, and manages context transitions before passing standard chat turns to `agent_runner`.

> **`kazma-core/kazma_core/state.py`**

- **Role:** Supervisor State Schema & Type Definitions
- **Architecture Links:** Standard state container used across all graph nodes in `graph_builder.py`.
- **Core Responsibilities & Operations:** Defines `SupervisorState` TypedDict containing turn history, scratchpad data, active tool calls, HITL approval status, model tracking, and error flags (`turn_failed`).

> **`kazma-core/kazma_core/time_travel.py`**

- **Role:** Conversation State Snapshot & Replay Engine
- **Architecture Links:** Persists state to SQLite WAL (`snapshots.db`); integrated with `graph_builder.py` and `graph.py` slash handlers.

- **Core Responsibilities & Operations:** Captures complete `SupervisorState` snapshots after supervisor steps, provides LRU-capped per-thread snapshot storage, and enables state rewind (`/replay`) and branching (`/fork`).

kazma-core/kazma_core/compaction.py

- **Role:** Conversation Context Window Compaction Engine
- **Architecture Links:** Invoked prior to LLM calls in `graph_builder.py` when token limits approach threshold.
- **Core Responsibilities & Operations:** Summarizes historical chat messages while preserving system prompts, tool results, and un-compacted recent turns to maintain token budget.

kazma-core/kazma_core/summarizer.py

- **Role:** Text Summarization & Context Reduction Service
- **Architecture Links:** Used by `compaction.py` , `stores/knowledge_ingest.py` , and memory consolidation routines.
- **Core Responsibilities & Operations:** Executes targeted prompt-driven text compression for long message threads, document chunks, and background job outputs.

2.2 Model Registry & LLM Provider Layer

kazma-core/kazma_core/model_registry.py

- **Role:** Multi-Provider Model Resolver & Client Factory
- **Architecture Links:** Central registry connecting all core services to `llm_provider.py` , `google_genai_provider.py` , `anthropic_llm.py` , `azure_llm.py` , and `bedrock_llm.py` .
- **Core Responsibilities & Operations:** Auto-corrects provider/model mismatches, maintains model capabilities mapping, resolves model fallback chains, and manages runtime profile switches via `set_active_model()` .

kazma-core/kazma_core/llm_provider.py

- **Role:** Generic OpenAI-Compatible Chat Completion Provider
- **Architecture Links:** Base provider implementation sending Bearer tokens to `/chat/completions` endpoints.
- **Core Responsibilities & Operations:** Handles standard chat completion HTTP calls, manages tool definitions, implements automatic 404 tool fallback, classifies errors into `transient` vs `permanent` via `LLMError` , and formats friendly error hints.

kazma-core/kazma_core/google_genai_provider.py

- **Role:** Native Google Gemini GenAI API Provider
- **Architecture Links:** Dispatched by `model_registry.py` for Google models; interfaces with Google `google-genai` SDK.

- **Core Responsibilities & Operations:** Constructs Gemini native requests, formats multimodal inputs, translates tool schemas, and manages Gemini-specific streaming responses.

`kazma-core/kazma_core/google_llm.py`

- **Role:** Google Vertex AI / PaLM Native LLM Interface
- **Architecture Links:** Enterprise Google Cloud backend extension for `model_registry.py`.
- **Core Responsibilities & Operations:** Authenticates via GCP service credentials, routes requests to Vertex AI endpoints, and formats response payloads.

`kazma-core/kazma_core/anthropic_llm.py`

- **Role:** Anthropic Messages API Provider
- **Architecture Links:** Dispatched by `model_registry.py` for Claude models.
- **Core Responsibilities & Operations:** Converts standard messages to Anthropic `/v1/messages` schema, injects `x-api-key` headers, handles system message separation, and parses tool call blocks.

`kazma-core/kazma_core/azure_llm.py`

- **Role:** Azure OpenAI Service Provider
- **Architecture Links:** Dispatched by `model_registry.py` for Azure deployments.
- **Core Responsibilities & Operations:** Handles Azure deployment IDs, formats `api-key` header authentication, injects `api-version` query parameters, and routes chat requests to Azure endpoints.

`kazma-core/kazma_core/bedrock_llm.py`

- **Role:** AWS Bedrock Runtime Provider
- **Architecture Links:** Dispatched by `model_registry.py` for AWS Bedrock models.
- **Core Responsibilities & Operations:** Sign AWS SigV4 requests, manages Bedrock Converse API request formatting, and translates tool calling structures for AWS Bedrock models.

`kazma-core/kazma_core/vision_capability.py`

- **Role:** Vision Capability Routing & Image Qualification Engine
- **Architecture Links:** Used by `tools/vision_analyze.py`, `kazma_gateway/agent_handler/attachments.py`, and `model_registry.py`.
- **Core Responsibilities & Operations:** Classifies models as vision-capable or text-only using deny/allow-lists, auto-selects dedicated vision clients when active model is text-only, and downgrades attached chat images to file stubs for non-vision models.

`kazma-core/kazma_core/models/router.py`

- **Role:** Heuristic Intent & Prompt Classification Router
- **Architecture Links:** Used by `models/selection.py` and `swarm/autoscaler.py`.

- **Core Responsibilities & Operations:** Analyzes prompt semantics to classify turn requirements (e.g., coding, deep reasoning, creative writing, fast response) for intelligent model selection.

> **kazma-core/kazma_core/models/selection.py**

- **Role:** Task-Optimal Model Selection Engine
- **Architecture Links:** Interacts with `model_registry.py`, `models/router.py`, and `config_store.py`.
- **Core Responsibilities & Operations:** Maps prompt intent classifications to configured model defaults, choosing the best model for a given task without mutating the global active model profile.

2.3 Configuration, Stores & Knowledge Base

> **kazma-core/kazma_core/config_store.py**

- **Role:** SQLite WAL-Backed Thread-Safe Application Configuration Store
- **Architecture Links:** Singleton service (`get_config_store()`) accessed by all packages for persistent dynamic settings.
- **Core Responsibilities & Operations:** Manages atomic reads and batch writes to `config.db`, auto-vault-encrypts sensitive keys, fires change listeners, and maintains runtime config state.

> **kazma-core/kazma_core/config_loader.py**

- **Role:** YAML File Configuration Ingestion Utility
- **Architecture Links:** Reads `kazma.yaml` and populates initial data in `config_store.py`.
- **Core Responsibilities & Operations:** Parses YAML structure, applies environment variable overrides, validates configuration schemas, and resolves default file paths.

> **kazma-core/kazma_core/config_schema.py**

- **Role:** Application Configuration Validation Schema
- **Architecture Links:** Pydantic/dataclass schema definitions used by `config_loader.py` and `config_store.py`.
- **Core Responsibilities & Operations:** Enforces type safety, default values, and structural validation across model configs, safety gates, and gateway options.

> **kazma-core/kazma_core/stores/workspaces.py**

- **Role:** Workspace Metadata & Repository Identity Store
- **Architecture Links:** Connected to `workspace/binding.py`, `ide/service.py`, and `ide/env_context.py`.
- **Core Responsibilities & Operations:** Persists workspace roots, cached Git remote metadata (`repo_url`, `owner`, `repo`, `default_branch`), active workspace selections, and triggers rebind notifications on repo switch.

`kazma-core/kazma_core/stores/knowledge.py`

- **Role:** Knowledge Base Document & Metadata Store
- **Architecture Links:** Interfaces with `kazma_ui/kb_api.py`, `stores/knowledge_chunker.py`, and `stores/knowledge_index.py`.
- **Core Responsibilities & Operations:** Manages document upload tracking, collection categorization, document status lifecycle, and record deletion.

`kazma-core/kazma_core/stores/knowledge_chunker.py`

- **Role:** Document Semantic Chunking Utility
- **Architecture Links:** Called by `stores/knowledge_ingest.py` during Knowledge Base document processing.
- **Core Responsibilities & Operations:** Splits raw text, Markdown, and PDF content into overlapping semantic text chunks optimized for vector embedding.

`kazma-core/kazma_core/stores/knowledge_index.py`

- **Role:** Vector Indexing & Hybrid Search Engine for Knowledge Base
- **Architecture Links:** Combines vector storage with SQLite FTS5 for RAG queries.
- **Core Responsibilities & Operations:** Generates embeddings for text chunks, indexes vector data, performs hybrid semantic + keyword search, and computes relevance scores.

`kazma-core/kazma_core/stores/knowledge_ingest.py`

- **Role:** Knowledge Base Ingestion Pipeline
- **Architecture Links:** Executed by background task queue via `stores/kb_jobs.py`.
- **Core Responsibilities & Operations:** Extracts plain text from multi-format files (PDF, DOCX, TXT, MD), orchestrates chunking, triggers vector indexing, and updates job completion status.

`kazma-core/kazma_core/stores/kb_jobs.py`

- **Role:** Async Knowledge Base Background Job Tracker
- **Architecture Links:** Connected to `kazma_ui/kb_api.py` and `stores/knowledge_ingest.py`.
- **Core Responsibilities & Operations:** Tracks ingestion job lifecycle (pending, processing, completed, failed), maintains error logs, and exposes progress status endpoints.

`kazma-core/kazma_core/stores/bookmarks.py`

- **Role:** User Conversation & Message Bookmark Persistence Store
- **Architecture Links:** Exposed to Web UI via `kazma_ui/routes_direct.py`.
- **Core Responsibilities & Operations:** Stores starred messages, tagged turns, and context notes in a dedicated SQLite database.

2.4 Swarm Orchestration Engine (`kazma-core/kazma_core/swarm/`)

`kazma-core/kazma_core/swarm/engine.py`

- **Role:** Central Swarm Orchestrator & Dispatch Controller
- **Architecture Links:** Primary entry point for multi-agent swarm tasks; coordinates `phonebook.py`, `reliability_registry.py`, `checkpoint_manager.py`, and `dispatch_inner.py`.
- **Core Responsibilities & Operations:** Manages task queues, delegates worker execution, enforces handoff cycle depth limits (max depth 5, max visits 2), handles pipeline checkpoints, and records swarm metrics.

`kazma-core/kazma_core/swarm/dispatch_inner.py`

- **Role:** Core Worker Dispatch Execution Branch
- **Architecture Links:** Called by `engine.py`; invokes `worker_dispatch.py` and `autoscaler.py`.
- **Core Responsibilities & Operations:** Executes the internal worker dispatch loop, triggers dynamic autoscaling on `NoCapableWorkersError`, and manages worker return payloads.

`kazma-core/kazma_core/swarm/dispatch_helpers.py`

- **Role:** Swarm Dispatch Context Formatting & State Transformation
- **Architecture Links:** Used by `dispatch_inner.py` and `handoff.py`.
- **Core Responsibilities & Operations:** Formats input messages, propagates metadata, prepares `SwarmDispatchContext`, and serializes intermediary worker responses.

`kazma-core/kazma_core/swarm/worker.py`

- **Role:** Base Swarm Worker Implementation
- **Architecture Links:** Inherited by `InProcessWorker`; connects worker execution to `llm_provider.py` and `ide/env_context.py`.
- **Core Responsibilities & Operations:** Formats worker system prompts with environment awareness, executes isolated LLM reasoning loops, invokes requested tools, and handles handoff triggers.

`kazma-core/kazma_core/swarm/worker_dispatch.py`

- **Role:** Isolated Worker Execution & Workspace Scope Wrapper
- **Architecture Links:** Wraps `worker.py` calls within `ide/workspace_scope.py`.
- **Core Responsibilities & Operations:** Applies per-task workspace scope `ContextVar`, ensures context isolation across concurrent workers, and dispatches worker execution.

`kazma-core/kazma_core/swarm/worker_factory.py`

- **Role:** Dynamic Worker Instantiation Factory
- **Architecture Links:** Consumed by `registry.py` and `autoscaler.py`.

- **Core Responsibilities & Operations:** Constructs fully initialized worker instances from template definitions, assigning tool registries and model configurations.

`kazma-core/kazma_core/swarm/reliability_registry.py`

- **Role:** Swarm Worker Reliability & Resiliency Manager
- **Architecture Links:** Extracted from `engine.py`; governs circuit breakers, retry policies, and timeout validators.
- **Core Responsibilities & Operations:** Tracks per-worker fault statistics, manages circuit breaker states (closed, open, half-open probe), enforces rate-limits, and isolates faulty worker nodes.

`kazma-core/kazma_core/swarm/reliability.py`

- **Role:** Circuit Breaker & Retry State Primitive Definitions
- **Architecture Links:** Primitive objects used by `reliability_registry.py`.
- **Core Responsibilities & Operations:** Implements half-open `_probe_in_flight` single-dispatch probe semantics, exponential backoff calculation, and failure threshold tracking.

`kazma-core/kazma_core/swarm/autoscaler.py`

- **Role:** Dynamic Swarm Worker Autoscaling Engine
- **Architecture Links:** Triggered on `NoCapableWorkersError` during swarm dispatch; reads `swarm_templates.json`.
- **Core Responsibilities & Operations:** Matches unhandled task tags against template expertise using token word-boundary matching, auto-spawns specialized worker instances, applies best-model selection, and reaps idle workers after timeout.

`kazma-core/kazma_core/swarm/checkpoint_manager.py`

- **Role:** Pipeline Checkpoint & HITL State Storage
- **Architecture Links:** Used by `engine.py` during PIPELINE execution pattern runs.
- **Core Responsibilities & Operations:** Pauses pipeline tasks at user-defined checkpoints, persists state to disk, handles manual approval/rejection, and enforces automatic rejection on timeout.

`kazma-core/kazma_core/swarm/phonebook.py`

- **Role:** Swarm Worker Topology & DAG Lookup Service
- **Architecture Links:** Used by `engine.py` for worker discovery and routing.
- **Core Responsibilities & Operations:** Resolves worker names, alias tags, and DAG execution pathways across active worker registrations.

`kazma-core/kazma_core/swarm/task_store.py`

- **Role:** SQLite WAL-Backed Durable Task Persistence Store
- **Architecture Links:** Stores task data for `engine.py`, `task_lifecycle.py`, and `kazma_ui/swarm_panel/`.

- **Core Responsibilities & Operations:** Manages task lifecycle storage, uses JSON expansion (`json_each()`) for exact worker filtering, handles schema migrations, and sets `busy_timeout=5000` .

> **`kazma-core/kazma_core/swarm/task_lifecycle.py`**

- **Role:** Task Transition & Status Lifecycle Management
- **Architecture Links:** Interacts with `task_store.py` and `engine.py` .
- **Core Responsibilities & Operations:** Enforces valid state machine transitions (pending → running → paused → completed/failed) for all swarm tasks.

> **`kazma-core/kazma_core/swarm/blackboard.py`**

- **Role:** Shared Inter-Worker Memory Blackboard
- **Architecture Links:** Accessed by concurrent swarm workers during complex multi-agent execution flows.
- **Core Responsibilities & Operations:** Provides thread-safe key-value data sharing, published observations, and structured findings across participating agents.

> **`kazma-core/kazma_core/swarm/sse_bridge.py`**

- **Role:** Real-time Swarm Event SSE Broadcaster
- **Architecture Links:** Connects `engine.py` events to `kazma_ui/swarm_sse.py` .
- **Core Responsibilities & Operations:** Formats and streams live agent handoffs, worker outputs, tool calls, and state changes to Web UI consumers via Server-Sent Events.

2.5 Cognitive Engine & V2 Memory Subsystem (`kazma-core/kazma_core/memory/`)

> **`kazma-core/kazma_core/memory/worker_bootstrap.py`**

- **Role:** V2 Memory Background Worker & Scheduler Bootstrapper
- **Architecture Links:** Invoked during `kazma_ui/app.py` startup.
- **Core Responsibilities & Operations:** Launches the background task queue worker, starts the 6-hour `macro_sleep` scheduler loop (decay & tier promotion), and starts the 24-hour backup/export scheduler loop.

> **`kazma-core/kazma_core/memory/task_queue.py`**

- **Role:** Durable Memory Maintenance Task Queue
- **Architecture Links:** Consumed by `worker_bootstrap.py` ; enqueues tasks from `consolidator.py` and `macro_sleep.py` .
- **Core Responsibilities & Operations:** Manages atomic SQLite task queueing, retry limits, and handler dispatch for background memory consolidation.

`kazma-core/kazma_core/memory/consolidator.py`

- **Role:** Episodic to Semantic Memory Consolidation Engine
- **Architecture Links:** Interacts with `belief_extractor.py`, `vector_engine.py`, and `graph_backend.py`.
- **Core Responsibilities & Operations:** Processes raw conversation logs, extracts structured facts and entities, updates memory stores, and triggers background consolidation tasks.

`kazma-core/kazma_core/memory/recall.py`

- **Role:** Context-Aware Hybrid Memory Recall Service
- **Architecture Links:** Invoked per chat turn in `agent_runner.py` and `sse_chat.py`.
- **Core Responsibilities & Operations:** Performs federated retrieval across current facts, vector similarity, knowledge graph edges (via PPR), and procedural memory rules to construct prompt injection context.

`kazma-core/kazma_core/memory/belief_extractor.py`

- **Role:** LLM-Driven Fact & Belief Extraction Parser
- **Architecture Links:** Used by `consolidator.py`.
- **Core Responsibilities & Operations:** Parses conversation turns to identify explicit user statements, preferences, world facts, and relational triples.

`kazma-core/kazma_core/memory/belief_mutation.py`

- **Role:** Memory Belief Conflict Resolution & Mutation Processor
- **Architecture Links:** Modifies records in `state_backend.py` and `graph_backend.py`.
- **Core Responsibilities & Operations:** Detects conflicting user facts, supersedes outdated statements, records assertion histories, and maintains belief coherence.

`kazma-core/kazma_core/memory/macro_sleep.py`

- **Role:** Periodic Memory Decay, Demotion & Re-consolidation Routine
- **Architecture Links:** Scheduled every 6 hours by `worker_bootstrap.py`.
- **Core Responsibilities & Operations:** Calculates memory recency/frequency decay scores, demotes inactive facts to archival tiers, promotes frequently accessed nodes, and purges transient records.

`kazma-core/kazma_core/memory/backup.py`

- **Role:** Native SQLite Memory Database Backup Routine
- **Architecture Links:** Scheduled every 24 hours by `worker_bootstrap.py`.
- **Core Responsibilities & Operations:** Performs hot online `sqlite3.backup()` of state and graph memory databases to backup storage directories without blocking active readers.

`kazma-core/kazma_core/memory/export.py`

- **Role:** Memory Graph & Fact Export Pipeline
- **Architecture Links:** Scheduled every 24 hours by `worker_bootstrap.py`.
- **Core Responsibilities & Operations:** Serializes active memory nodes, relationships, and belief histories to standardized JSONL and GraphML file formats.

`kazma-core/kazma_core/memory/vector_engine.py`

- **Role:** Multi-Backend Vector Similarity Search Engine
- **Architecture Links:** Used by `recall.py` and `consolidator.py`; supports `sqlite-vec` and fallback NumPy implementations.
- **Core Responsibilities & Operations:** Embeds text snippets, stores high-dimensional vector representations, and executes fast top-k cosine similarity queries.

`kazma-core/kazma_core/memory/graph_backend.py`

- **Role:** Knowledge Graph Relationship Persistence Store
- **Architecture Links:** Stores entity-relationship triples for `ppr.py` and `recall.py`.
- **Core Responsibilities & Operations:** Manages entity nodes, typed edges, relationship weights, and graph traversal queries in SQLite.

`kazma-core/kazma_core/memory/ppr.py`

- **Role:** Personalized PageRank Graph Traversal Algorithm
- **Architecture Links:** Used by `recall.py` for contextual graph spreading activation.
- **Core Responsibilities & Operations:** Computes Personalized PageRank vectors starting from query seed entities to surface relevant multi-hop graph context.

`kazma-core/kazma_core/memory/federated_search.py`

- **Role:** Unified Multi-Source Memory Search Aggregator
- **Architecture Links:** Combines results from `vector_engine.py`, `graph_backend.py`, and `current_facts.py`.
- **Core Responsibilities & Operations:** Executes parallel search queries across disparate memory backends and merges/ranks results using reciprocal rank fusion (RRF).

`kazma-core/kazma_core/memory/schema_v2.py`

- **Role:** V2 Memory Subsystem Database DDL & Migration Schema
- **Architecture Links:** Applied by `state_backend.py` and `graph_backend.py` during initialization.
- **Core Responsibilities & Operations:** Defines table schemas, index structures, foreign key constraints, and automatic column migrations for memory SQLite stores.

2.6 IDE Subsystem & Workspace Management (`kazma-core/kazma_core/ide/`)

`kazma-core/kazma_core/ide/service.py`

- **Role:** Transport-Agnostic Workspace & File System Operations Backend
- **Architecture Links:** Accessed by `kazma_ui/ide_api.py` , `kazma_tui/editor.py` , and `/ide` chat commands.
- **Core Responsibilities & Operations:** Executes safe file reading/writing, tree listing, line-range editing, terminal process execution, and Git operations while routing all mutating actions through `LocalToolRegistry.execute()` .

`kazma-core/kazma_core/ide/env_context.py`

- **Role:** System Prompt Workspace Awareness Builder
- **Architecture Links:** Injected at agent startup in `agent_runner.py` , per turn in `kazma_ui/sse_chat.py` , and in swarm worker prompts (`worker.py`).
- **Core Responsibilities & Operations:** Constructs Markdown environment blocks containing active workspace root paths, Git branch status, remote URLs, GitHub auth status, and available system tools.

`kazma-core/kazma_core/ide/workspace_scope.py`

- **Role:** Async Task Workspace Target Scoping Manager
- **Architecture Links:** ContextVar-backed async context manager used by `swarm/worker_dispatch.py` .
- **Core Responsibilities & Operations:** Isolates workspace target root bindings per async task execution, allowing concurrent tasks to operate safely on different repositories.

`kazma-core/kazma_core/workspace/binding.py`

- **Role:** Single Source of Truth Workspace Root Resolver
- **Architecture Links:** Used across all `file_*` tools, `ide/service.py` , and `stores/workspaces.py` .
- **Core Responsibilities & Operations:** Evaluates workspace binding ladder precedence (`workspace_scope` ContextVar → active `WorkspaceStore` row → static pin → `KAZMA_WORKSPACE` env → default sandbox path) and notifies rebind listeners.

`kazma-core/kazma_core/workspace/mcp_rebind.py`

- **Role:** MCP Server Dynamic Workspace Binding Handler
- **Architecture Links:** Triggered by `workspace/binding.py` `notify_root_changed()` .
- **Core Responsibilities & Operations:** Dynamically updates workspace-bound Model Context Protocol (MCP) server process arguments and environment variables when the active repository changes.

2.7 Safety, Human-In-The-Loop (HITL) & Security

`kazma-core/kazma_core/safety/hitl.py`

- **Role:** Canonical Human-In-The-Loop Security Gate Manager
- **Architecture Links:** Integrated into `graph_builder.py:tool_worker_node` and consumed by `get_hitl_config()`.
- **Core Responsibilities & Operations:** Defines `CANONICAL_DANGER_TOOLS`, intercepts tool execution for high-risk operations (e.g., code execution, file modification, system installs), and issues LangGraph `interrupt()` commands awaiting user approval.

`kazma-core/kazma_core/safety/hitl_grants.py`

- **Role:** Temporary HITL Approval Grant & TTL Manager
- **Architecture Links:** Interacts with `safety/hitl.py` and Web/Gateway approval endpoints.
- **Core Responsibilities & Operations:** Stores temporary single-turn or time-bound tool approval grants so approved tools can run without repeated prompting.

`kazma-core/kazma_core/safety/prompt_fence.py`

- **Role:** Prompt Injection Defense & Data Fencing Utility
- **Architecture Links:** Applied in `skills/self_improvement.py` and untrusted content injection paths.
- **Core Responsibilities & Operations:** Evaluates input strings for system prompt override markers and wraps untrusted user/tool outputs in `<kazma:data untrusted>` isolation blocks.

`kazma-core/kazma_core/safety/yolo.py`

- **Role:** Unattended Automated Execution ("YOLO") TTL Switch
- **Architecture Links:** Used by `safety/hitl.py` and `tools/registry.py`.
- **Core Responsibilities & Operations:** Manages temporary bypass timers for HITL approval gates during explicitly configured fully automated bulk operations.

`kazma-core/kazma_core/security/vault.py`

- **Role:** Sensitive Credential Vault & Encryption Service
- **Architecture Links:** Used by `config_store.py` and `kazma_skills/native/secret_vault/`.
- **Core Responsibilities & Operations:** Encrypts sensitive configuration items (API keys, tokens, passwords) at rest using AES-GCM / Fernet encryption schemes.

`kazma-core/kazma_core/security/platform_rbac.py`

- **Role:** Role-Based Access Control Enforcer
- **Architecture Links:** Applied across gateway channel commands and UI routes.
- **Core Responsibilities & Operations:** Validates user permissions against configured platform roles (Admin, Developer, Viewer) before executing administrative or workspace commands.

kazma-core/kazma_core/security/ssrf.py

- **Role:** Server-Side Request Forgery Security Filter
- **Architecture Links:** Used by `tools/read_url.py` and `web_acquire/fetch.py`.
- **Core Responsibilities & Operations:** Validates outbound URLs against local loopback IP ranges, private subnet CIDRs, and AWS metadata IP endpoints (`169.254.169.254`) to prevent SSRF exploitation.

kazma-core/kazma_core/security/linter.py

- **Role:** System Code & Tool Output Security Linter
- **Architecture Links:** Used prior to tool execution and document generation.
- **Core Responsibilities & Operations:** Scans code blocks for hardcoded secrets, dangerous subprocess calls, and malformed command syntax.

kazma-core/kazma_core/security/audit_trail.py

- **Role:** Security Event Audit Logging Service
- **Architecture Links:** Records events across HITL approvals, credential accesses, and role checks.
- **Core Responsibilities & Operations:** Writes structured, tamper-evident audit log entries for all security-sensitive system operations.

2.8 Scraping Proxy Layer (`kazma-core/kazma_core/proxy/`)

kazma-core/kazma_core/proxy/registry.py

- **Role:** Proxy Provider Dynamic Registry & Resolver
- **Architecture Links:** Re-reads `proxy.provider` dynamically on fetch operations; returns `NullProvider` on error.
- **Core Responsibilities & Operations:** Manages pluggable proxy implementations (AnyIP, BrightData, Oxylabs) and provides live dynamic instance resolution for web tools.

kazma-core/kazma_core/proxy/client.py

- **Role:** Scraping HTTP Client Factory
- **Architecture Links:** Used exclusively by web acquisition tools (`read_url.py` , `web_search.py` , `crawl.py`).
- **Core Responsibilities & Operations:** Constructs `httpx.AsyncClient` instances configured with proxy routing and random User-Agent rotation. (Does NOT route LLM provider traffic).

kazma-core/kazma_core/proxy/anyip.py

- **Role:** AnyIP Residential Rotating Proxy Integration
- **Architecture Links:** Registered in `proxy/registry.py` .

- **Core Responsibilities & Operations:** Formats authentication credentials and endpoints for routing web requests through AnyIP residential IP pools.

> `kazma-core/kazma_core/proxy/base.py`

- **Role:** Abstract Proxy Provider Interface
- **Architecture Links:** Base class for all proxy provider modules.
- **Core Responsibilities & Operations:** Defines standard `get_proxy_url()` interface and health checking methods.

2.9 Built-in Tools & Agent Capabilities (`kazma-core/kazma_core/tools/`)

> `kazma-core/kazma_core/tools/registry.py`

- **Role:** Central Tool Registration & Execution Manager
- **Architecture Links:** Connected to `graph_builder.py` , `ide/service.py` , and `safety/hitl.py` .
- **Core Responsibilities & Operations:** Registers all available system and skill tools, exposes unified tool schemas to LLM providers, and routes tool execution through safety check gates (`safety.check()`).

> `kazma-core/kazma_core/tools/file_read.py`

- **Role:** Workspace File Reading Tool Implementation
- **Architecture Links:** Invoked by agents; uses `workspace/binding.py` .
- **Core Responsibilities & Operations:** Reads workspace file contents with line offset slicing, utf-8 decoding, and path containment validation.

> `kazma-core/kazma_core/tools/file_write.py`

- **Role:** Workspace File Modification Tool Implementation
- **Architecture Links:** Invoked by agents; routes through `LocalToolRegistry.execute()` .
- **Core Responsibilities & Operations:** Creates, overwrites, or applies targeted string diff modifications to workspace files within path boundaries.

> `kazma-core/kazma_core/tools/code_exec.py`

- **Role:** Sandboxed Command & Script Execution Tool
- **Architecture Links:** Invoked by agents for shell execution; guarded by HITL gates.
- **Core Responsibilities & Operations:** Runs shell commands in configured workspace directories, enforces timeout limits, captures stdout/stderr, and returns execution codes.

> `kazma-core/kazma_core/tools/read_url.py`

- **Role:** Web Page Content Extraction Tool
- **Architecture Links:** Uses `proxy/client.py` and `security/ssrf.py` .

- **Core Responsibilities & Operations:** Fetches external HTTP/HTTPS pages, filters SSRF target IPs, strips HTML markup into clean Markdown, and handles scraping retries.

`kazma-core/kazma_core/tools/web_search.py`

- **Role:** Multi-Engine Web Search Tool Implementation
- **Architecture Links:** Connects agents to SearXNG, DuckDuckGo, and Google search services.
- **Core Responsibilities & Operations:** Formats search queries, queries search APIs, parses web results, and formats structured citation snippets for LLM consumption.

`kazma-core/kazma_core/tools/web_research.py`

- **Role:** Autonomous Deep Web Research Workflow Tool
- **Architecture Links:** Coordinates `web_search.py`, `read_url.py`, and `research_planner.py`.
- **Core Responsibilities & Operations:** Executes multi-step research plans, crawls relevant source links, extracts evidentiary passages, and synthesizes analytical summaries.

`kazma-core/kazma_core/tools/research_planner.py`

- **Role:** Deep Research Sub-Task Decomposition Planner
- **Architecture Links:** Used by `web_research.py` and `kazma_ui/research_panel/`.
- **Core Responsibilities & Operations:** Decomposes complex user queries into structured execution graphs containing targeted web queries and evaluation criteria.

`kazma-core/kazma_core/tools/research_synthesize.py`

- **Role:** Multi-Source Research Findings Synthesis Tool
- **Architecture Links:** Used in final step of research pipeline.
- **Core Responsibilities & Operations:** Merges extracted evidence blocks, reconciles contradictory source claims, and generates comprehensive markdown reports with explicit source citations.

`kazma-core/kazma_core/tools/image_gen.py`

- **Role:** Multi-Backend AI Image Generation Tool
- **Architecture Links:** Dispatches to sub-modules in `tools/image_backends/`.
- **Core Responsibilities & Operations:** Routes image generation requests across DALL-E 3, Flux, Stability AI, and Pollinations backends, saving output artifacts locally.

`kazma-core/kazma_core/tools/vision_analyze.py`

- **Role:** Image & Visual Asset Analysis Tool
- **Architecture Links:** Interacts with `vision_capability.py` and `model_registry.py`.
- **Core Responsibilities & Operations:** Inspects local image files, selects a vision-capable LLM client, and generates detailed visual descriptions or code transcriptions.

2.10 Voice & Web Acquisition (`voice/` and `web_acquire/`)

`kazma-core/kazma_core/voice/stt.py`

- **Role:** Speech-To-Text Audio Transcription Service
- **Architecture Links:** Connected to `kazma_gateway/agent_handler.py` (voice notes) and `kazma_ui/routes_voice.py`.
- **Core Responsibilities & Operations:** Transcribes incoming audio files (OGG, WAV, MP3) to plain text using OpenAI Whisper or local STT models.

`kazma-core/kazma_core/voice/tts.py`

- **Role:** Text-To-Speech Speech Synthesis Engine
- **Architecture Links:** Used by gateway voice responses and Web UI voice streaming endpoints.
- **Core Responsibilities & Operations:** Synthesizes textual agent replies into spoken audio streams via ElevenLabs, Edge-TTS, or OpenAI TTS engines.

`kazma-core/kazma_core/voice/vad.py`

- **Role:** Voice Activity Detection Processing Engine
- **Architecture Links:** Used by real-time WebSocket audio streaming routes (`routes_voice_ws.py`).
- **Core Responsibilities & Operations:** Processes real-time audio chunk buffers to detect speech start/stop boundaries and suppress background silence.

`kazma-core/kazma_core/web_acquire/crawl.py`

- **Role:** Recursive Web Crawler Utility
- **Architecture Links:** Uses `web_acquire/fetch.py` and `proxy/client.py`.
- **Core Responsibilities & Operations:** Recursively crawls domain URL trees up to configured depth limits, respecting robots.txt and rate limits.

`kazma-core/kazma_core/web_acquire/fetch.py`

- **Role:** Low-Level Web Document Fetcher
- **Architecture Links:** Uses `proxy/client.py` and `security/ssrf.py`.
- **Core Responsibilities & Operations:** Downloads single URL documents, handles charset encodings, follows redirect limits, and converts HTML DOM structures to Markdown.

`kazma-core/kazma_core/web_acquire/search.py`

- **Role:** Multi-Provider Search Provider Adapter
- **Architecture Links:** Connects to SearXNG, Tavily, and Google Search engines.
- **Core Responsibilities & Operations:** Normalizes search parameters and unifies raw search JSON responses into standardized result objects.

2.11 System Maintenance, Cron & Observability

`kazma-core/kazma_core/system/installer.py`

- **Role:** System Tool & Dependency Installer Service
- **Architecture Links:** Used by environment bootstrapper skills and system management APIs.
- **Core Responsibilities & Operations:** Checks system PATH for required binaries (`git` , `node` , `ripgrep` , `uv`) and automates missing tool installation across supported platforms.

`kazma-core/kazma_core/system/maintenance.py`

- **Role:** System Cleanup & Database Hygiene Service
- **Architecture Links:** Scheduled periodically or invoked via CLI/UI.
- **Core Responsibilities & Operations:** Truncates expired session records, purges temporary scratch files, reclaims SQLite WAL space (`VACUUM`), and prunes stale task logs.

`kazma-core/kazma_core/system/runtime_manager.py`

- **Role:** Process & Subsystem Lifecycle Controller
- **Architecture Links:** Monitored by `kazma_ui/app.py` and `kazma_cli` .
- **Core Responsibilities & Operations:** Tracks runtime status of background threads, worker pools, database connection pools, and registered MCP server processes.

`kazma-core/kazma_core/cron/scheduler.py`

- **Role:** Persistent Cron Job Scheduler Engine
- **Architecture Links:** Interacts with `kazma_gateway` and `agent_runner.py` .
- **Core Responsibilities & Operations:** Parses cron schedule expressions, stores scheduled jobs in SQLite, and fires automated message prompts into supervisor queues.

`kazma-core/kazma_core/observability/alerts.py`

- **Role:** System Alert & Exception Notification Handler
- **Architecture Links:** Listens to `llm_provider.py` , `swarm/engine.py` , and database handlers.
- **Core Responsibilities & Operations:** Generates system alert payloads on persistent failures, high error rates, or security violations, dispatching alerts to configured admin webhooks.

`kazma-core/kazma_core/tracing/events.py`

- **Role:** Distributed Tracing Event Telemetry Logger
- **Architecture Links:** Connected to `swarm/tracing.py` and `kazma_ui/telemetry_route.py` .
- **Core Responsibilities & Operations:** Emits standardized trace span events covering supervisor turns, tool execution durations, and LLM call latencies.

`kazma-core/kazma_core/telemetry.py`

- **Role:** Usage Metrics & System Performance Aggregator

- **Architecture Links:** Exposed via `kazma_ui/telemetry_route.py` and `scripts/generate_metrics.py`.
- **Core Responsibilities & Operations:** Aggregates token consumption metrics, cost estimations, turn execution counts, and API latency histograms.

---

3. Kazma Multi-Platform Gateway (`kazma-gateway/kazma_gateway/`)

`kazma-gateway/kazma_gateway/gateway_app.py`

- **Role:** Gateway Subsystem Multi-Channel Bootstrapper
- **Architecture Links:** Entry point for `kazma-gateway`; initializes channel adapters in `adapters/`.
- **Core Responsibilities & Operations:** Loads channel bot tokens from `config_store.py`, instantiates enabled adapters (Telegram, Discord, Slack), registers callback routes, and starts polling/webhook loops.

`kazma-gateway/kazma_gateway/agent_handler.py`

- **Role:** Platform Isolation & Session Target Resolution Handler
- **Architecture Links:** Crucial boundary between messaging platforms and `kazma_core/agent_runner.py`.
- **Core Responsibilities & Operations:** Strips platform-specific identifiers (`chat_id`, `message_id`) from graph states, resolves target destinations via `SessionStore`, attaches incoming media, and formats outgoing agent replies.

`kazma-gateway/kazma_gateway/session_store.py`

- **Role:** Chat Platform Session Mapping Store
- **Architecture Links:** Used by `agent_handler.py` and `commands.py`.
- **Core Responsibilities & Operations:** Persists mappings between internal thread IDs and platform specific chat context (`platform`, `chat_id`, `user_id`, `reply_to_message_id`).

`kazma-gateway/kazma_gateway/commands.py`

- **Role:** Gateway Slash Command Parser & Interceptor
- **Architecture Links:** Intercepts platform command messages (`/start`, `/help`, `/model`, `/ide`, `/swarm`, `/hitl`).
- **Core Responsibilities & Operations:** Handles gateway command execution, updates session settings, manages user authorization, and delegates complex commands to `kazma_core/graph.py`.

`kazma-gateway/kazma_gateway/post_hitl_shell.py`

- **Role:** Gateway Interactive Approval Command Shell
- **Architecture Links:** Handles user responses to HITL approval prompts sent via chat platforms.

- **Core Responsibilities & Operations:** Processes inline button callbacks or textual approval/denial commands (`/hitl approve` , `/hitl deny`), unblocking pending graph interrupts.

> **`kazma-gateway/kazma_gateway/adapters/telegram.py`**

- **Role:** Telegram Bot API Platform Adapter
- **Architecture Links:** Inherits `adapters/base.py` ; communicates with Telegram Bot API via `python-telegram-bot` .
- **Core Responsibilities & Operations:** Handles long-polling/webhook updates, formats MarkdownV2 responses, renders inline keyboard approval buttons, and processes voice notes.

> **`kazma-gateway/kazma_gateway/adapters/discord.py`**

- **Role:** Discord Platform Bot Adapter
- **Architecture Links:** Inherits `adapters/base.py` ; communicates with Discord Gateway API via `discord.py` .
- **Core Responsibilities & Operations:** Listens to Discord server channels and DMs, splits responses over 2000 characters, renders message component buttons, and processes attachments.

> **`kazma-gateway/kazma_gateway/adapters/slack.py`**

- **Role:** Slack Platform App Adapter
- **Architecture Links:** Inherits `adapters/base.py` ; uses Slack Bolt SDK.
- **Core Responsibilities & Operations:** Processes Slack events and slash commands, formats Block Kit messages, manages thread replies, and handles interactive button payloads.

> **`kazma-gateway/kazma_gateway/adapters/fanout_bus.py`**

- **Role:** Multi-Platform HITL Approval Fan-Out Adapter
- **Architecture Links:** Used when multiple messaging platforms are configured concurrently.
- **Core Responsibilities & Operations:** Fans out HITL approval requests to all active admin channel adapters simultaneously; applies first-approval-wins resolution logic across platforms.

---

4. Kazma Web UI Subsystem (`kazma-ui/kazma_ui/`)

4.1 FastAPI Backend Router & Server

> **`kazma-ui/kazma_ui/app.py`**

- **Role:** Main FastAPI Web Application & API Entry Point
- **Architecture Links:** Serves HTML templates, static assets, and mounts API sub-routers (`sse_chat` , `ide_api` , `swarm_panel` , `kb_api` , `memory_api`).

- **Core Responsibilities & Operations:** Configures CORS middleware, initializes database connection pools, compiles global graph holders, bootstraps memory background workers, and defines root web routes.

`kazma-ui/kazma_ui/sse_chat.py`

- **Role:** Real-time Server-Sent Events Chat Streaming Controller
- **Architecture Links:** Primary chat streaming API endpoint (`POST /api/chat/stream`); calls `kazma_core/agent_runner.py` .
- **Core Responsibilities & Operations:** Injects `env_context` , streams graph tokens, tool execution events, time-travel snapshots, and HITL interrupts to the frontend chat UI via SSE.

`kazma-ui/kazma_ui/ide_api.py`

- **Role:** Web IDE Backend REST API Router
- **Architecture Links:** Exposes `kazma_core/ide/service.py` to the frontend `ide.js` module.
- **Core Responsibilities & Operations:** Provides workspace file tree endpoints, file content reads/writes, terminal command execution, Git status/commit operations, and swarm delegation endpoints.

`kazma-ui/kazma_ui/kb_api.py`

- **Role:** Knowledge Base Management API Router
- **Architecture Links:** Interfaces with `kazma_core/stores/knowledge.py` and `stores/knowledge_ingest.py` .
- **Core Responsibilities & Operations:** Handles document upload requests, knowledge base search queries, collection management, and ingestion job status polling.

`kazma-ui/kazma_ui/memory_api.py`

- **Role:** Cognitive Memory Console API Router
- **Architecture Links:** Exposes `kazma_core/memory/recall.py` , `consolidator.py` , and `graph_backend.py` .
- **Core Responsibilities & Operations:** Exposes endpoints for querying memory facts, searching entity relationship graphs, triggering manual re-consolidation, and inspecting belief histories.

`kazma-ui/kazma_ui/saas_api.py`

- **Role:** SaaS Multi-Tenancy & User Account API Router
- **Architecture Links:** Interacts with `security/platform_rbac.py` and user authentication middleware.
- **Core Responsibilities & Operations:** Manages user authentication, session cookies, organization workspaces, usage quotas, and tenant isolation context.

`kazma-ui/kazma_ui/models_route.py`

- **Role:** Model Configuration & Provider Settings Router

- **Architecture Links:** Interacts directly with `kazma_core/model_registry.py` and `config_store.py`.
- **Core Responsibilities & Operations:** Exposes endpoints for listing available models, testing provider API key connectivity, updating default model assignments, and switching active profiles.

> **`kazma-ui/kazma_ui/settings.py`**

- **Role:** Global System Settings REST Router
- **Architecture Links:** Connects Web UI settings panels to `kazma_core/config_store.py`.
- **Core Responsibilities & Operations:** Reads and updates system configurations including proxy parameters, voice options, HITL approval rules, and gateway bot tokens.

> **`kazma-ui/kazma_ui/hitl_approval.py`**

- **Role:** Web UI HITL Approval Endpoint
- **Architecture Links:** Receives approval actions from `static/js/hitl_approval.js` and resumes pending graph interrupts in `agent_runner.py`.
- **Core Responsibilities & Operations:** Handles `POST /api/approve/{thread_id}` calls, resumes paused supervisor turns with approval/rejection flags, and updates execution grants.

> **`kazma-ui/kazma_ui/health.py`**

- **Role:** System Health Check & Readiness Endpoint
- **Architecture Links:** Probed by load balancers, Docker health checks, and status dashboards.
- **Core Responsibilities & Operations:** Checks connectivity to SQLite WAL databases, PostgreSQL connection pools, vector stores, and active LLM provider endpoints.

4.2 Web UI Frontend Controllers & Templates

> **`kazma-ui/kazma_ui/static/js/app.js`**

- **Role:** Frontend Master Application Initialization & Navigation
- **Architecture Links:** Loaded on all HTML pages; sets up global toast notifications, Alpine.js stores, and theme toggles.
- **Core Responsibilities & Operations:** Initializes global event buses, configures HTMX default headers, manages active tab states, and handles global socket reconnects.

> **`kazma-ui/kazma_ui/static/js/chat.js`**

- **Role:** Web Chat Interface Event Controller
- **Architecture Links:** Connects `templates/chat.html` to `kazma_ui/sse_chat.py`.
- **Core Responsibilities & Operations:** Handles user input submission, parses incoming SSE event streams, renders Markdown formatted messages, displays active tool execution cards, and renders HITL approval modals.

`kazma-ui/kazma_ui/static/js/ide.js`

- **Role:** Web IDE Editor Component Controller
- **Architecture Links:** Communicates with `kazma_ui/ide_api.py`.
- **Core Responsibilities & Operations:** Renders interactive workspace file trees, manages multi-tab code editor buffers, connects integrated terminal buffers, and handles Git operations.

`kazma-ui/kazma_ui/static/js/swarm.js`

- **Role:** Swarm Panel Dashboard Controller
- **Architecture Links:** Consumes real-time event streams from `kazma_ui/swarm_sse.py`.
- **Core Responsibilities & Operations:** Renders live agent handoff DAG graphs, displays worker node load meters, tracks active swarm task queues, and handles task dispatch forms.

`kazma-ui/kazma_ui/static/js/memory.js`

- **Role:** Memory Console Visualization Controller
- **Architecture Links:** Communicates with `kazma_ui/memory_api.py`.
- **Core Responsibilities & Operations:** Visualizes knowledge graph entity connections, displays active user facts, presents memory recency/frequency metrics, and provides manual fact edit controls.

`kazma-ui/kazma_ui/static/css/kazma.css`

- **Role:** System-Wide Styling & Design Tokens stylesheet
- **Architecture Links:** Applied across all Jinja2 HTML templates in `templates/`.
- **Core Responsibilities & Operations:** Defines CSS variables for dark/light themes, typography scales, glassmorphism card components, animation Keyframes, and responsive layouts.

`kazma-ui/kazma_ui/templates/index.html`

- **Role:** Main Dashboard Landing Page Template
- **Architecture Links:** Extends `templates/base.html`.
- **Core Responsibilities & Operations:** Displays high-level system metrics, active session summaries, recent swarm activity, and quick navigation links.

`kazma-ui/kazma_ui/templates/chat.html`

- **Role:** Web AI Chat Application View Template
- **Architecture Links:** Rendered by `kazma_ui/app.py`; powered by `static/js/chat.js`.
- **Core Responsibilities & Operations:** Provides main chat conversation layout, prompt input textareas, message history scroll containers, attachment dropzones, and tool output inspectors.

`kazma-ui/kazma_ui/templates/ide.html`

- **Role:** Integrated Development Environment View Template

- **Architecture Links:** Rendered by `kazma_ui/app.py`; powered by `static/js/ide.js`.
- **Core Responsibilities & Operations:** Layout container for file sidebar, multi-file code editor tabs, AI workspace side-chat, and bottom terminal/Git drawer.

> **`kazma-ui/kazma_ui/templates/swarm.html`**

- **Role:** Swarm Multi-Agent Management View Template
- **Architecture Links:** Rendered by `kazma_ui/app.py`; powered by `static/js/swarm.js`.
- **Core Responsibilities & Operations:** Displays active worker node cards, pipeline task creation forms, DAG execution visualization canvases, and live swarm log feeds.

---

5. Kazma Text User Interface (TUI) Subsystem (`kazma-tui/kazma_tui/`)

> **`kazma-tui/kazma_tui/app.py`**

- **Role:** Textual Terminal User Interface Application Root
- **Architecture Links:** Entry point for `kazma-tui`; connects to `kazma_core`.
- **Core Responsibilities & Operations:** Configures Textual application layout, binds global keyboard shortcuts, manages screen navigation, and applies CSS themes.

> **`kazma-tui/kazma_tui/editor.py`**

- **Role:** Interactive TUI Code Editor Screen
- **Architecture Links:** Consumes `kazma_core/ide/service.py`.
- **Core Responsibilities & Operations:** Provides full-screen terminal file editing with syntax highlighting, line numbers, diff previews, and direct AI coding assistance.

> **`kazma-tui/kazma_tui/files.py`**

- **Role:** TUI Workspace File Explorer Widget
- **Architecture Links:** Interacts with `kazma_core/ide/service.py`.
- **Core Responsibilities & Operations:** Renders interactive terminal directory tree views, allows file selection, and triggers file opens in `editor.py`.

> **`kazma-tui/kazma_tui/chat.py`**

- **Role:** TUI AI Chat View Screen
- **Architecture Links:** Connects terminal input to `kazma_core/agent_runner.py`.
- **Core Responsibilities & Operations:** Displays streaming chat turns in terminal buffer, formats Markdown code blocks, and presents interactive tool confirmation prompts.

> **`kazma-tui/kazma_tui/swarm.py`**

- **Role:** TUI Swarm Orchestration View Screen

- **Architecture Links:** Communicates with `kazma_core/swarm/engine.py`.
- **Core Responsibilities & Operations:** Displays live ASCII status maps of swarm workers, pending task tables, worker CPU/memory usage, and task log streams.

> **`kazma-tui/kazma_tui/dashboard.py`**

- **Role:** TUI Overview Dashboard Screen
- **Architecture Links:** Pulls system stats from `kazma_core/telemetry.py`.
- **Core Responsibilities & Operations:** Renders active session counts, token consumption sparklines, current model profiles, and gateway connectivity indicators.

> **`kazma-tui/kazma_tui/widgets/hitl_modal.py`**

- **Role:** TUI HITL Tool Approval Modal Dialog
- **Architecture Links:** Triggered when graph interrupt occurs in terminal chat turns.
- **Core Responsibilities & Operations:** Displays high-risk tool call details (command string, file diff, target path) and captures keypresses (`[y] approve / [n] deny`).

> **`kazma-tui/kazma_tui/widgets/model_picker.py`**

- **Role:** TUI Interactive Model Picker Widget
- **Architecture Links:** Interacts with `kazma_core/model_registry.py`.
- **Core Responsibilities & Operations:** Allows developers to filter and switch active LLM provider models directly from the terminal interface.

---

6. Kazma Command Line Interface (`kazma-cli/kazma_cli/`)

> **`kazma-cli/kazma_cli/main.py`**

- **Role:** Primary CLI Command Parser & Entry Point
- **Architecture Links:** Configures `kazma` binary entry point using `Click` or `Typer`.
- **Core Responsibilities & Operations:** Dispatches subcommand execution (`kazma start`, `kazma gateway`, `kazma swarm`, `kazma project`, `kazma update`).

> **`kazma-cli/kazma_cli/gateway.py`**

- **Role:** CLI Gateway Management Subcommand
- **Architecture Links:** Controls `kazma_gateway/gateway_app.py`.
- **Core Responsibilities & Operations:** Starts, stops, and inspects status of Telegram, Discord, and Slack gateway bot processes from terminal commands.

> **`kazma-cli/kazma_cli/swarm.py`**

- **Role:** CLI Swarm Control & Inspection Subcommand

- **Architecture Links:** Interacts with `kazma_core/swarm/engine.py`.
- **Core Responsibilities & Operations:** Lists active swarm workers, dispatches tasks from local files, inspects task store queues, and reaps stalled instances.

`kazma-cli/kazma_cli/project.py`

- **Role:** CLI Project Initialization & Workspace Utility
- **Architecture Links:** Generates project boilerplate structures.
- **Core Responsibilities & Operations:** Scaffolds new Kazma agent project structures, creates default `kazma.yaml` files, and validates local directory permissions.

`kazma-cli/kazma_cli/update.py`

- **Role:** CLI System Update Controller
- **Architecture Links:** Interfaces with Git repository and `pyproject.toml`.
- **Core Responsibilities & Operations:** Checks for latest framework releases, pulls remote updates, executes schema migrations, and updates package installations via `uv`.

---

7. Kazma Skills Framework (`kazma-skills/kazma_skills/`)

`kazma-skills/kazma_skills/native_loader.py`

- **Role:** Native Skill Discovery & Ingestion Loader
- **Architecture Links:** Discovers skills in `kazma_skills/native/` and registers tools with `kazma_core/tools/registry.py`.
- **Core Responsibilities & Operations:** Scans skill manifests (`skill_manifest.yaml`), validates tool definitions, injects configuration parameters, and registers tool callable instances.

`kazma-skills/kazma_skills/manifest.py`

- **Role:** Skill Manifest Validation Schema Parser
- **Architecture Links:** Used by `native_loader.py`.
- **Core Responsibilities & Operations:** Validates skill manifest YAML files for required metadata (name, description, version, required environment variables, tool definitions).

`kazma-skills/kazma_skills/native/git_github_manager/tools.py`

- **Role:** Git & GitHub Operations Native Skill
- **Architecture Links:** Uses shared `GitHubClient` and local `git` CLI; registered in tool registry.
- **Core Responsibilities & Operations:** Executes pull requests creation, issue tracking, commit pushing, branch switching, and diff generation.

kazma-skills/kazma_skills/native/email_manager/tools.py

- **Role:** Universal Email Management Native Skill
- **Architecture Links:** Supports Gmail API, Microsoft Graph, and standard IMAP/SMTP backends in `email_manager/backends/`.
- **Core Responsibilities & Operations:** Performs email searching, thread reading, draft composition, attachment extraction, and message sending.

kazma-skills/kazma_skills/native/secret_vault/tools.py

- **Role:** Credential Secret Vault Native Skill
- **Architecture Links:** Interacts with `kazma_core/security/vault.py`.
- **Core Responsibilities & Operations:** Allows authorized agents to securely retrieve, store, and update encrypted API keys and credentials.

kazma-skills/kazma_skills/native/browser_automation/tools.py

- **Role:** Playwright Browser Automation Native Skill
- **Architecture Links:** Integrates Playwright headless browser instance with agent tool registry.
- **Core Responsibilities & Operations:** Executes complex web browser interactions including clicking elements, filling forms, taking visual screenshots, and executing JavaScript.

kazma-skills/kazma_skills/native/code_analyzer_linter/tools.py

- **Role:** Codebase Static Analysis & Linting Skill
- **Architecture Links:** Used by agents and IDE subsystem.
- **Core Responsibilities & Operations:** Runs language linters (`flake8` , `mypy` , `eslint`), extracts diagnostic syntax errors, and reports structured fix suggestions.

kazma-skills/kazma_skills/native/database_client/tools.py

- **Role:** Multi-Database Query Native Skill
- **Architecture Links:** Connects to PostgreSQL, MySQL, and SQLite databases.
- **Core Responsibilities & Operations:** Executes parameterized SQL queries, inspects database table schemas, and returns structured query result tables.

---

8. Operational & Maintenance Scripts (`scripts/`)

scripts/backup_kazma.py

- **Role:** System State Backup Utility Script
- **Architecture Links:** Backs up SQLite databases (`config.db` , `memory.db` , `snapshots.db` , `tasks.db`).

- **Core Responsibilities & Operations:** Creates timestamped compressed archive backups of system state databases and configuration files.

`scripts/restore_kazma.py`

- **Role:** System State Restoration Utility Script
- **Architecture Links:** Restores data produced by `backup_kazma.py`.
- **Core Responsibilities & Operations:** Validates backup archive integrity, safely replaces database files, and executes required schema migrations upon restore.

`scripts/migrate_sqlite_to_postgres.py`

- **Role:** Database Backend Migration Script
- **Architecture Links:** Migrates data from local SQLite stores to enterprise PostgreSQL databases.
- **Core Responsibilities & Operations:** Reads all records from SQLite WAL databases, transforms schemas to PostgreSQL conventions, and performs bulk database inserts.

`scripts/generate_metrics.py`

- **Role:** System Metrics & Performance Report Generator
- **Architecture Links:** Consumes data from `kazma_core/telemetry.py`.
- **Core Responsibilities & Operations:** Analyzes token usage logs, latency statistics, and error rates, rendering Markdown and HTML metric summary reports.

`scripts/reembed.py`

- **Role:** Memory Vector Re-embedding Script
- **Architecture Links:** Interacts with `kazma_core/memory/vector_engine.py`.
- **Core Responsibilities & Operations:** Re-generates high-dimensional vector embeddings across all stored memory facts when changing embedding model providers.

`scripts/smoke_production.py`

- **Role:** Production Readiness Automated Verification Suite
- **Architecture Links:** Probes `kazma-ui`, `kazma-gateway`, LLM providers, and memory subsystems.
- **Core Responsibilities & Operations:** Executes end-to-end smoke tests against live deployments to confirm API readiness, streaming coherence, and system stability.

`scripts/start-web.sh`

- **Role:** Web Server Production Launch Daemon
- **Architecture Links:** Invoked in container environments and production systemd units.
- **Core Responsibilities & Operations:** Sets up production environment variables, configures Gunicorn/Uvicorn worker process counts, and launches `serve.py`.

Summary Matrix

Package	Directory Path	Primary Responsibility
`kazma-core`	`kazma-core/kazma_core/`	Agent runner, LangGraph state engine, LLM providers, model registry, swarm engine, V2 memory subsystem, IDE service, safety HITL gates, proxy layer, built-in tools.
`kazma-gateway`	`kazma-gateway/kazma_gateway/`	Multi-channel platform adapters (Telegram, Discord, Slack), chat session isolation, gateway commands, HITL shell callbacks.
`kazma-ui`	`kazma-ui/kazma_ui/`	FastAPI web application backend, SSE chat streaming endpoints, Web IDE endpoints, Swarm dashboard panel, Knowledge Base & Memory management APIs, static JS/CSS frontend, Jinja2 templates.
`kazma-tui`	`kazma-tui/kazma_tui/`	Textual terminal dashboard, interactive code editor screen, ASCII swarm status visualizers, HITL terminal approval modals.
`kazma-cli`	`kazma-cli/kazma_cli/`	Command line management tool (`kazma`) for system controls, gateway orchestration, swarm task dispatches, and project initialization.
`kazma-memory`	`kazma-memory/kazma_memory/`	Arabic tokenization utilities and search backend modules.
`kazma-skills`	`kazma-skills/kazma_skills/`	Extensible native agent skills (Git/GitHub manager, Email client, Secret vault, Browser automation, Linter, DB client).
`scripts`	`scripts/`	Backup/restore scripts, SQLite-to-Postgres migration tools, metric generation, vector re-embedding, and production smoke tests.